TechNova Support Bot: Observability Report

LangChain + LangGraph + NVIDIA NIM + LangSmith | Prepared by Sruthi Arukala | LangSmith project: technova-bot-sruthi

1. Summary for the team lead

I built a customer-support chatbot for a fictional electronics store. It looks up orders, answers policy questions and remembers the conversation. I then measured how fast it is, what it costs, how it fails and how good its answers are, using two NVIDIA-hosted models (small: **openai/gpt-oss-20b**, large: **nvidia/nemotron-3-super-120b-a12b**). **Recommendation: deploy the small model** with a time limit and retry, and keep the large model as a backup. It is about 7 times cheaper and scored the same on quality. Details are in section 4.

Metric	Small (gpt-oss-20b)	Large (nemotron-120b)
Latency p50 (20 questions)	1.66 s	1.45 s
Latency p95 (20 questions)	4.96 s	2.77 s
Latency average	2.25 s	1.70 s
Average tokens per conversation	about 792	about 1,236
Estimated monthly cost (10,000 chats a day)	about $47	about $334
Error rate, normal requests	0% (0 of 20)	0% (0 of 20)
Error rate, forced-failure test	30% (6 of 20, by design)	not run
Evaluation PASS rate (10 questions, AI judge)	0.8	0.8
Answer quality (my judgement, 1 to 5)	4	4

Tokens per conversation = benchmark tokens (14,248 and 22,241) divided by the 18 chats in the 20-question set. Costs use the assumed rate card from the assignment (small $0.20, large $0.90 per 1M tokens), so they are estimates, and the short test chats probably make them lower than real life. The forced-failure test used the small model.

2. Key findings

- **The model call is the slow part.** In the slowest run (6.08 s) the model took 6.07 s. My own code and the routing took about 0 s. A similar refusal took only 0.69 s, so the length of the answer was not the cause. A busy shared server is a likely explanation (not proven). **Fix:** a time limit with retry, built in the error-handling tests.
- **The large model was not slower here.** Each model was timed once, so one slow moment moves the numbers a lot. Speed depends on more than model size.
- **Streaming:** the first word appeared at 2.10 s on average and the full answer at 2.61 s (5 questions). The total time is the same, but the user sees text about half a second earlier, so it feels faster.
- **Tokens grow with every turn** because the whole history is sent again on every call. Keeping only the last 6 messages cut input tokens from 11,600 to 6,324 (about 45%), but the bot then forgot older facts (it named the wrong first order on turn 9). Fewer tokens means less memory.
- **A strict rulebook matters.** In the first test the bot invented a return label and a free replacement. Stricter rules fixed it, and each rule change had to be tested again.

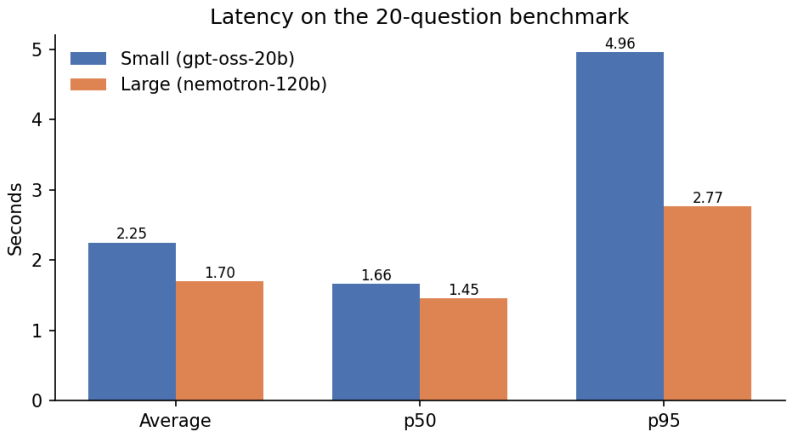


Figure 1. Latency of the two models on the 20-question benchmark (one run each).

3. Evidence from LangSmith

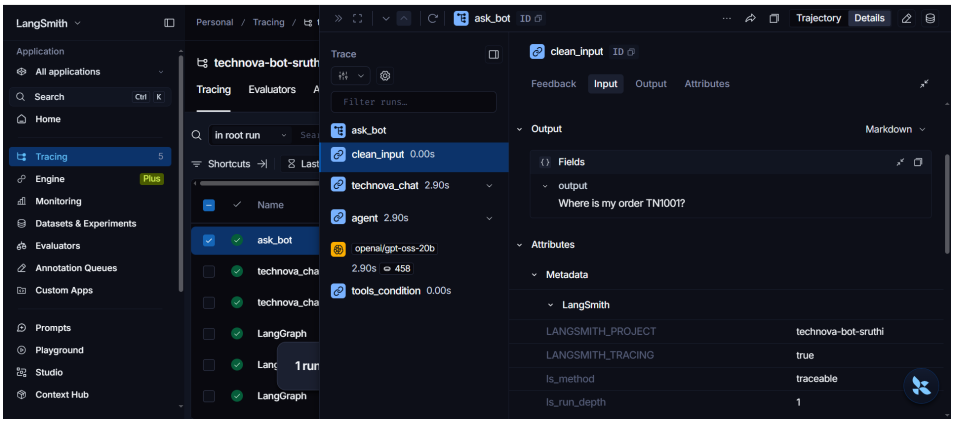


Figure 2. Run tree: ask_bot contains our own traced helper clean_input (0.00 s) and the bot (technova_chat, 2.90 s). The model call, openai/gpt-oss-20b, used 458 tokens.

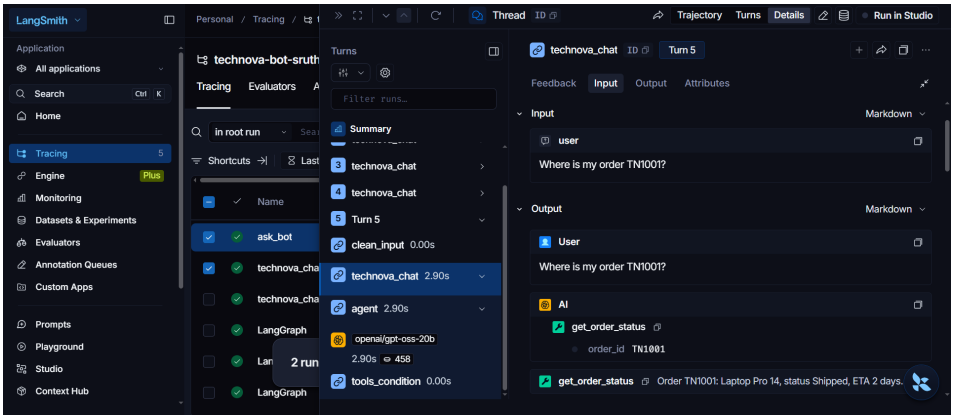


Figure 3. Thread: the runs of one conversation are grouped by thread_id. The turn shown asks for order TN1001, the bot calls get_order_status and the tool returns the order.

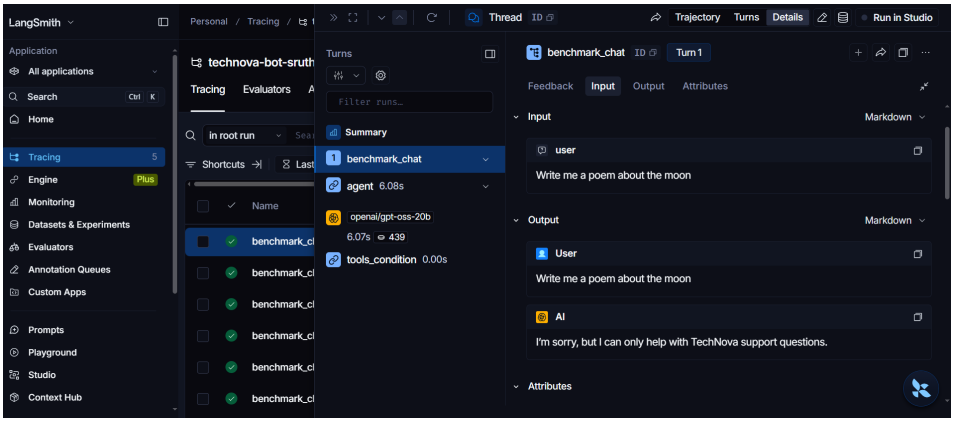


Figure 4. Slowest run (6.08 s): agent 6.08 s, model call 6.07 s, tools_condition 0.00 s. The model call is the slow step.

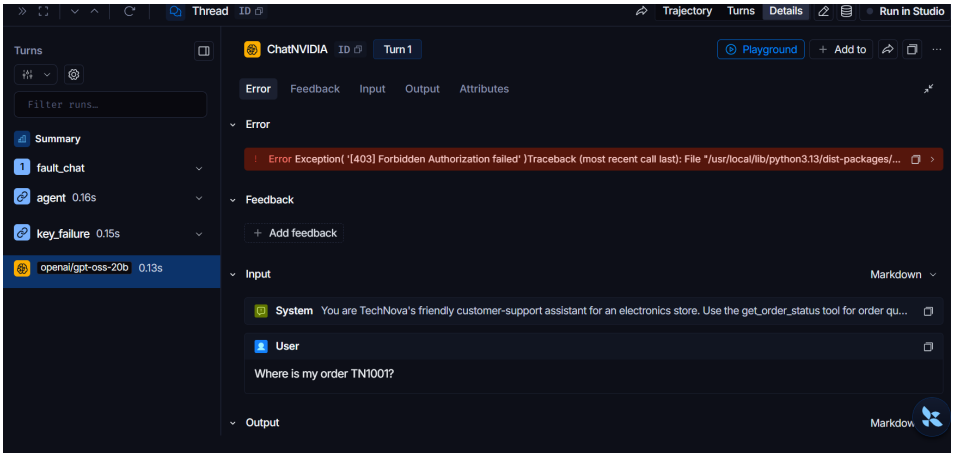


Figure 5. Failed run: wrong API key. The model call returns 403 Forbidden (red Error tab). The bot made one attempt only (agent 0.16 s) and did not retry, and the user got a clear message.

4. Error handling, evaluation and recommendation

Error handling. I broke the bot on purpose in five ways. The user never sees a Python error.

Scenario	Fix	What the user sees
Wrong model name	Fallback to a backup model	The backup answers normally
Wrong API key	Fail fast: 1 attempt, no retry	"Our assistant is not set up correctly right now..."
Slow or failed call	Retry up to 3 times with backoff	The answer, or "taking longer than usual" if all fail
Tool failure	Error goes back to the model	An apology: order system temporarily unavailable
Bad input (empty, too long)	Checked before the model is called	Guidance, and no tokens are used

Error rate: 6 errors in 20 requests = 30%. This is high because 6 of the 20 requests were broken on purpose (2 bad input, 1 bad key, 1 timeout, 1 tool failure, 1 bad model). The 14 normal requests had 0 errors. A retry makes the user wait longer (the fixed timeout case took about 7 s), so a production system also needs a time limit.

Evaluation. 10 questions with reference answers, marked PASS or FAIL by an AI judge (the large model). Both models scored **0.8**, a tie. With 10 questions I cannot separate them, and the judge made some mistakes (for example one blank verdict), so a human should spot-check the marks. My first judge version gave 0.3 and 0.5 because I read its answer wrongly (order IDs contain the digits 1 and 0); I fixed it and use the PASS/FAIL version. 5 benchmark runs received a thumbs-up rating (user_rating = 1).



Figure 6. Evaluation comparison of small-v2 (A) and large-v2 (B). Correctness is the same for both (about 0.8). Latency in this view is higher than in the benchmark because the exam script adds a 2-second pause before every question.

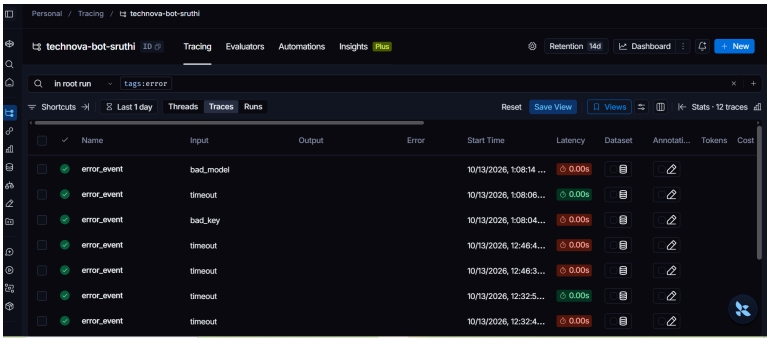


Figure 7. Error counter in LangSmith: error_event rows (tag error), one per handled error, with the error type in each row. 12 rows are shown for the last day, which includes the earlier single-scenario tests. The 20-request test itself logged 6 errors (results/error_rate.csv).

5. Recommendation: deploy the small model

- **Cost:** about $47 against $334 a month at 10,000 chats a day, roughly 7 times cheaper, because it uses fewer tokens and each token is cheaper.
- **Quality:** both scored 0.8 in the evaluation and 4 out of 5 in my own review, so I found no quality reason to pay more.
- **Latency:** the large model was faster in this test (p95 2.77 s against 4.96 s). The small model's slow cases are the risk, so I would add a 6-second time limit with retry, and use the large model as a fallback.
- **When to change my mind:** if the real target is a p95 under 3 seconds, or if a larger evaluation shows the small model failing more often, the large model is worth its price. The tests behind this recommendation are small (one latency run per model, 10 evaluation questions).

6. Top three risks to monitor, with alert thresholds

These thresholds are my proposals, based on the baselines I measured. They should be tuned after a week of real traffic.

Risk	Why it matters (evidence)	Alert rule
1. Slow answers	p95 was 4.96 s on the small model, and one run took 6.08 s because the model call was slow.	Alert if p95 latency is above 8 s for 5 minutes.
2. Errors and rate limits	NVIDIA answered 429 Too Many Requests when I went fast, and a wrong key gives 403. Normal traffic had 0 errors, so any rise is a real signal.	Alert if the error rate is above 5% over 10 minutes. Alert at once on any 403 (key problem).
3. Token and cost growth	Input tokens grow with every turn, and the baseline is about 792 tokens per conversation (about $1.58 a day at 10,000 chats).	Alert if average tokens per conversation are above 1,600 (twice the baseline) or the daily cost is above $3.

Also worth watching: answer quality. Re-run the evaluation each week and alert if the PASS rate falls below 0.7.

7. What I did and what I learned

- **Built** a LangGraph bot with two tools, a rulebook and memory, and traced every run in LangSmith with tags, metadata and my own traced function.
- **Measured** latency (p50, p95, time to first word), tokens, cost and error rate, and saved the numbers as CSV files in the results folder.
- **Found and fixed problems:** an invented refund process, fake fast times caused by 429 errors, a wrongly marked AI judge, and memory lost by trimming.
- **Models:** the Llama 8B and 70B models named in the assignment had been retired by NVIDIA. A model appearing in the catalogue does not mean it works, so I tested models first and used the two that answered.
- **Limits:** one latency run per model, a 10-question evaluation, assumed prices, and no bonus items (chat UI, dashboard and alerts, guardrail node).